MATPMD4 Assignment 1 (Stochastic Processes)

Student ID: 3539054
Module: MATPMD4
Date: March 2026

Instructions & Integrity

I confirm that I have read and understood the instructions on the cover page.

In [ ]:
```
# Required Imports
# !pip install numpy matplotlib
import numpy as np
import matplotlib.pyplot as plt

# Set precision for cleaner output
np.set_printoptions(precision=4, suppress=True)
```

Question 2

2(a) Calculating x

Reasoning: The matrix P is a **stochastic matrix**. A fundamental property of stochastic matrices is that the sum of probabilities in any given row must equal exactly 1. Mathematically, this is because the process must transition to *some* state (including potentially staying in the same state) in the next time step.

$$\sum_j P_{ij} = 1 \quad \forall i$$

For the third row (State C), we have:

$$0.16 + 0.16 + 0.22 + x + 0.26 = 1$$

$$0.80 + x = 1 \implies x = 0.2$$

In [2]:
```
# Row C is [0.16, 0.16, 0.22, x, 0.26]
# Stochastic matrix property: sum of probabilities in a row must equal 1.
x = 1.0 - (0.16 + 0.16 + 0.22 + 0.26)
x = round(x, 2)  # Round to avoid floating point errors

print(f"\nCalculated x: {x}")

# Define the Transition Matrix P
# Rows: A, B, C, D, E
P = np.array([
    [0.21, 0.07, 0.15, 0.11, 0.46],
    [0.00, 1.00, 0.00, 0.00, 0.00],
```

```
    [0.16, 0.16, 0.22,    x, 0.26],
    [0.00, 0.00, 0.00, 1.00, 0.00],
    [0.21, 0.27, 0.18, 0.24, 0.10]
])

print("\nTransition Matrix P:\n", P)
```

```
Calculated x: 0.2

Transition Matrix P:
 [[0.21 0.07 0.15 0.11 0.46]
 [0.   1.   0.   0.   0.  ]
 [0.16 0.16 0.22 0.2  0.26]
 [0.   0.   0.   1.   0.  ]
 [0.21 0.27 0.18 0.24 0.1 ]]
```

State Transition Diagram

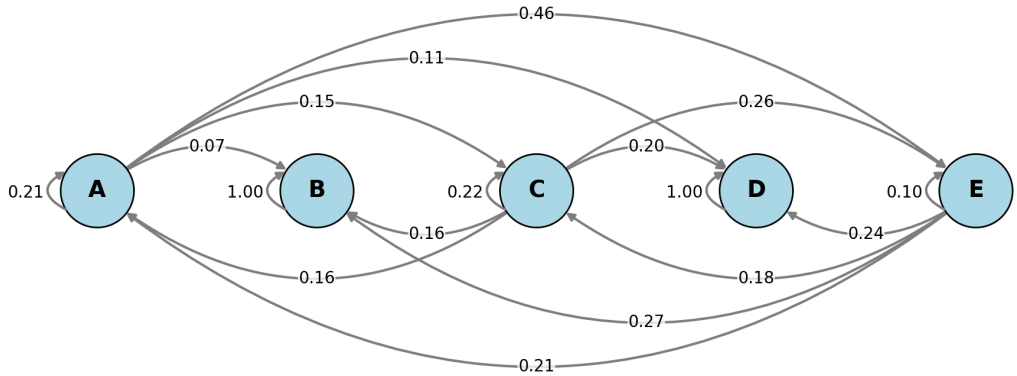

Figure 1: State transition diagram showing transition probabilities P_{ij} for states A-E.

2(b) Distribution after 3 Generations

Observation:
As the number of generations increases, the probability values in the state vector **pi** change significantly:

- The probabilities for states **A** and **C** are consistently **decreasing**.
- The probability for state **E increases initially** (due to inflow from A and C) before eventually **decreasing**.
- The probabilities for states **B** and **D** are consistently **increasing**.

Explanation:
This behavior is expected because **B** and **D** are **absorbing states**.

- Looking at the transition matrix P (and the diagram), states **B** and **D** have a **self-loop probability of 1.0** ($P_{BB} = 1, P_{DD} = 1$).
- This means once the process enters state B or D, it cannot leave.

- States **A**, **C**, and **E** are **transient states**, meaning the process will eventually leave them and get trapped in B or D.
- Therefore, as $n \to \infty$, the total probability of being in **A, C, or E drops to 0**, while the total probability **accumulates** entirely into states **B** and **D**.

In [3]:
```
# Initial distribution: A(13%), B(24%), C(32%), D(28%), E(3%)
pi_0 = np.array([0.13, 0.24, 0.32, 0.28, 0.03])
print(f"Gen 0 Distribution: {pi_0}")

# Generation 1
# Using @ for matrix multiplication
pi_1 = pi_0 @ P
print(f"\nGen 1 Distribution: {pi_1}")

# Generation 2
pi_2 = pi_1 @ P
print(f"\nGen 2 Distribution: {pi_2}")

# Generation 3
pi_3 = pi_2 @ P
print(f"\nGen 3 Distribution: {pi_3}")
```

```
Gen 0 Distribution: [0.13 0.24 0.32 0.28 0.03]

Gen 1 Distribution: [0.0848 0.3084 0.0953 0.3655 0.146 ]

Gen 2 Distribution: [0.0637 0.369  0.06   0.4289 0.0784]

Gen 3 Distribution: [0.0394 0.4042 0.0369 0.4667 0.0527]
```

2(c) Canonical Form

We rewrite P in canonical form by reordering the states so that absorbing states appear first, followed by transient states.

- **Absorbing States:** B, D
- **Transient States:** A, C, E

The canonical form is partitioned as:

$$P = \begin{pmatrix} I & 0 \\ R & Q \end{pmatrix}$$

Where Q represents transitions between transient states, and R represents transitions from transient to absorbing states.

In [4]:
```
# Current State Order: A(0), B(1), C(2), D(3), E(4)
# Absorbing States: B(1), D(3)
# Transient States: A(0), C(2), E(4)

# The new order to group Absorbing first, then Transient.
# New Order: B, D, A, C, E
canonical_order = [1, 3, 0, 2, 4]

# Move Row B to top & Row D to second to top
P_rows_sorted = P[canonical_order, :]
```

```python
# Move Col B to left, Col D to second to left
P_canonical = P_rows_sorted[:, canonical_order]

print("Canonical Matrix P_canonical:\n", P_canonical)

# Structure of Canonical Form:
# | I  0 |  (Absorbing rows)
# | R  Q |  (Transient rows)

# 2 absorbing states and 3 transient states.
num_absorbing = 2
num_transient = 3

# Q: Transient to Transient (Bottom-Right block)
# Take rows from index 2 to end, and columns from index 2 to end
Q = P_canonical[num_absorbing:, num_absorbing:]

print("\nMatrix Q (Transient -> Transient):\n", Q)

# R: Transient to Absorbing (Bottom-Left block)
# Take rows from index 2 to end, and columns from index 0 to 2
R = P_canonical[num_absorbing:, :num_absorbing]

print("\nMatrix R (Transient -> Absorbing):\n", R)
```

```
Canonical Matrix P_canonical:
 [[1.   0.   0.   0.   0.  ]
 [0.   1.   0.   0.   0.  ]
 [0.07 0.11 0.21 0.15 0.46]
 [0.16 0.2  0.16 0.22 0.26]
 [0.27 0.24 0.21 0.18 0.1 ]]

Matrix Q (Transient -> Transient):
 [[0.21 0.15 0.46]
 [0.16 0.22 0.26]
 [0.21 0.18 0.1 ]]

Matrix R (Transient -> Absorbing):
 [[0.07 0.11]
 [0.16 0.2 ]
 [0.27 0.24]]
```

2(d) Fundamental Matrix N

The fundamental matrix N gives the expected number of times the process is in transient state j, given it started in transient state i.

Formula:

$$N = (I - Q)^{-1}$$

Components:

- I: Identity matrix of size equivalent to the number of transient states (3x3).
- Q: The matrix describing probabilities of moving between transient states (*A*, *C*, *E*).

In [5]:
```
I = np.eye(3)
N = np.linalg.inv(I - Q)

print("Fundamental Matrix N:\n", N)
```

```
Fundamental Matrix N:
 [[1.6412 0.5456 0.9964]
 [0.4975 1.539  0.6989]
 [0.4824 0.4351 1.4834]]
```

2(e) Mean Time to Absorption

This calculates the expected number of steps before the process hits *any* absorbing state, given it starts in a specific transient state.

Formula:

$$M = N\mathbf{1}$$

Components:

- N: The fundamental matrix calculated in 2(d).
- $\mathbf{1}$: A column vector of ones.
- Summing the rows of N (which is what multiplying by $\mathbf{1}$ does) adds up the expected visits to *all* transient states, giving the total lifetime of the process.

In [6]:
```
# Formula: M = N @ 1 (column vector of ones)
ones = np.ones((3, 1))
M = N @ ones

print("Mean steps to absorption (from A, C, E):")
print(M)
```

```
Mean steps to absorption (from A, C, E):
[[3.1832]
 [2.7353]
 [2.4009]]
```

2(f) Absorption Probabilities

This calculates the probability that the process will be absorbed into a specific absorbing state (B or D).

Formula:

$$B = NR$$

Components:

- N: The fundamental matrix representing time spent in transient states.
- R: The matrix of probabilities of moving from a transient state directly to an absorbing state.
- Multiplying N by R aggregates the probabilities of all paths leading to a specific absorbing state.

In [7]:
```python
# Formula: B = N @ R
B = N @ R

print("Absorption Probabilities (Rows: A, C, E | Cols: B, D):")
print(B)
```

```
Absorption Probabilities (Rows: A, C, E | Cols: B, D):
[[0.4712 0.5288]
 [0.4698 0.5302]
 [0.5039 0.4961]]
```